autolisp-runtime Design

Codex

June 24, 2026

Contents

1 Purpose

This document defines the initial runtime object model for `clautolisp`.

It also records the first explicit handoff from `autolisp-reader` objects to runtime values.

2 Design Position

The runtime shall not collapse AutoLISP semantics onto raw Common Lisp objects indiscriminately.

However, not every AutoLISP value needs a wrapper.

The current design uses a mixed strategy:

- signed 32-bit integers map directly to Common Lisp integers,
- reals map directly to Common Lisp `double-float` values,
- conses and the empty list map directly to Common Lisp conses and `nil`,
- all other runtime-visible AutoLISP objects use explicit Common Lisp structures.

This gives a simple and efficient representation for the most common literal data while preserving control over the types that materially differ from Common Lisp.

3 Immediate Mappings

The following AutoLISP runtime values map directly to Common Lisp objects:

3.1 NIL

- AutoLISP `nil` maps to Common Lisp `nil`.
- It remains both false and the empty list.
- It is not modeled as a separate wrapper object.

3.2 INT

- AutoLISP integers map to Common Lisp integers.
- The intended runtime contract is signed 32-bit, even though Common Lisp may physically represent them with a broader integer implementation.

3.3 REAL

- AutoLISP reals map to Common Lisp `double-float`.

3.4 LIST and dotted pairs

- AutoLISP proper lists and dotted pairs map to Common Lisp cons structure.
- Elements and tails are runtime values, not reader objects.

4 Wrapped Runtime Objects

The following AutoLISP-visible values shall use explicit runtime structures:

4.1 Symbol

AutoLISP symbols must not be represented by raw Common Lisp package symbols.

Initial structure responsibilities:

- canonical symbol name,
- optional original reader spelling,
- future property data,
- identity independent from Common Lisp packages.

Symbols are interned in a dedicated runtime symbol table.

Formal design rule:

- an AutoLISP symbol is an identity of name, not a container for namespace-local value or function cells,
- value and function bindings are attached to namespaces and dynamic frames, not to the symbol object itself,
- the same symbol object may therefore appear in multiple namespaces and in dynamic variable bindings without carrying those bindings internally.

4.2 String

AutoLISP strings use a wrapper around a Common Lisp `string`.

Current rationale:

- keep room for AutoLISP-specific behavior without overloading raw Common Lisp strings,
- keep a consistent representation policy for runtime-visible non-immediate objects,
- make the runtime type model explicit for `type` and related predicates.

The wrapper stores at least:

- a `string-value` slot containing the Common Lisp string.

4.3 Pathnames

AutoLISP pathnames are represented as strings at the language level.

Current design position:

- pathname arguments and results remain ordinary AutoLISP string objects,
- there is no separate runtime pathname value type at this stage,
- pathname resolution is **not** delegated implicitly to Common Lisp pathname merging semantics,
- the pathname resolution algorithm itself must come from the AutoLISP specification and compatibility evidence, not from ad hoc implementation choice.

Instead, path resolution belongs to explicit runtime / host state interpreted through an AutoLISP-defined resolution algorithm.

That state will need to define at least:

- a current working directory or equivalent current path prefix,
- host-profile-specific separator, drive, and volume conventions,
- any AutoLISP-visible search-path or document-directory behavior that affects relative path resolution.

Implementation rule for future file builtins:

- do not pass unresolved relative AutoLISP pathname strings directly to Common Lisp file operators,
- either resolve them first to an absolute host path under AutoLISP rules, or otherwise bind Common Lisp pathname defaults to a benign value at the boundary,
- avoid letting `*default-pathname-defaults*` silently define AutoLISP-visible path behavior.

This keeps pathname merging and current-directory semantics under explicit project control instead of inheriting accidental host Lisp behavior.

The remaining open work is therefore twofold:

- determine the actual AutoLISP pathname-resolution algorithm from the spec and product-compatibility corpus,
- then implement that algorithm over explicit runtime / host path state.

4.4 FILE

File descriptors are wrapper objects around Common Lisp streams.

The wrapper should eventually store at least:

- host stream,
- open path,
- open mode,
- host profile metadata if needed.

4.5 ENAME, PICKSET, VLA-object, VARIANT, SAFEARRAY, reactor-like objects

These are host-managed or compatibility-layer values and should all be wrapped explicitly around host or emulation-layer payloads.

The first implementation only establishes generic wrapper shapes so the runtime type model can grow without redesigning the boundary.

5 Reader Handoff

The reader produces source-aware reader objects.

The runtime consumes runtime values.

The handoff is therefore an explicit mapping step:

1. `autolisp-reader` parses text into reader objects.
2. `autolisp-runtime` maps reader objects into runtime values.
3. later evaluator layers consume those runtime values.

Initial mapping rules:

- `symbol-object` -> interned `autolisp-symbol`
- `string-object` -> `autolisp-string`
- `integer-object` -> signed integer
- `real-object` -> `double-float`
- `cons-object` -> Common Lisp cons / list of mapped elements

- `quote-object` -> a list whose first element is the interned symbol `QUOTE`

Comment objects and concrete-syntax-only objects do not enter ordinary runtime mapping.

6 Formal Environment Model

This section records the first formal environment model for `clautolisp`.

It is intentionally conservative and tracks only what is justified by the current AutoLISP / Visual LISP specification corpus.

6.1 Core Entities

The runtime environment model is built from the following distinct entity classes:

- `autolisp-symbol` name identity only
- `value-cell` a storage location for a variable value plus bound/unbound state
- `function-cell` a storage location for a callable binding plus defined/undefined state
- `namespace` a mapping from symbols to value cells and/or function cells
- `dynamic-frame` a temporary mapping from symbols to variable bindings active during evaluation
- `evaluation-context` the current session, current document, current namespace, and active dynamic frames used to evaluate a form

6.2 Symbol Rule

An `autolisp-symbol` denotes only a name identity.

It does **not** contain:

- a current variable value,
- a current function definition,
- any namespace-local binding state.

Consequently:

- the same symbol object may be referenced by multiple document namespaces,
- the same symbol object may be temporarily rebound in a dynamic frame,
- modifying a binding updates a cell in a namespace or frame, not the symbol object itself.

6.3 Namespaces

At least the following namespace classes are required by the current specification corpus:

- **document namespace** namespace associated with a drawing/document
- **blackboard namespace** shared namespace used for inter-document communication
- **separate VLX namespace** namespace used by a separate-namespace VLX application

Each document namespace contains at least:

- a mapping from symbols to value cells,
- a mapping from symbols to function cells.

The blackboard namespace contains at least:

- a mapping from symbols to value cells.

The separate VLX namespace contains at least:

- a mapping from symbols to function cells,
- and possibly a mapping from symbols to value cells where required by the namespace functions.

The exact completeness rules for VLX namespaces remain under-specified and must be refined against Autodesk and BricsCAD behavior.

Current runtime/session rule:

- a runtime session keeps an explicit registry of document namespaces by name,
- the session owns one shared blackboard namespace,
- selecting or entering a current document registers that document in the session registry,
- the session also keeps a set of propagated symbols for conservative cross-document value copying.

6.4 Dynamic Frames

Dynamic frames are used for temporary **variable** bindings created during evaluation.

Current design rule:

- dynamic frames affect variable lookup only,
- dynamic frames do **not** currently provide an independent dynamic function binding layer,
- no separate function-binding stack shall be assumed unless required by later specification evidence.

Therefore, the current model does **not** postulate an AutoLISP analogue of a dynamic `flet` environment by default.

6.5 Callable Objects and Special Operators

The runtime currently distinguishes two callable object classes:

- `SUBR` builtin or primitive function implemented directly in Common Lisp
- `USUBR` user-defined AutoLISP function evaluated by the AutoLISP evaluator

Current design rule:

- `SUBR` wraps a Common Lisp function together with an AutoLISP-visible name,

- `USUBR` stores an AutoLISP lambda list, body forms, and definition metadata used by the evaluator,
- special operators are **not** represented as `SUBR` objects; they are handled by an explicit evaluator dispatch table over raw forms.

6.6 Current Namespace and Current Document

The evaluation context contains at least:

- the current session,
- the current document,
- the current namespace of evaluation,
- the active dynamic-frame stack.

The **current document** is modeled as dynamic execution context supplied by the host entry point, not as a lexical binding.

Current rule:

- when the host invokes AutoLISP evaluation for a given drawing, it establishes the current document in the evaluation context,
- ordinary function calls inherit that current document,
- only host-driven document switches or explicit namespace-transfer mechanisms may change it.

The runtime now exposes this boundary explicitly through host-entry helpers:

- a runtime session has a current document,
- constructing an evaluation context without overrides defaults to that session document and uses it as the current namespace,
- entering a document context can explicitly select a distinct VLX namespace while still naming the current document.

The **current namespace** is the namespace whose ordinary variable and function cells are used by default for evaluation.

Current rule:

- in ordinary document-scoped evaluation, the current namespace is the current document namespace,
- in separate-namespace VLX evaluation, the current namespace may be a VLX namespace distinct from the current document namespace,
- explicit namespace functions such as `vl-doc-ref`, `vl-doc-set`, `vl-bb-ref`, `vl-bb-set`, `vl-doc-export`, and `vl-doc-import` cross these default boundaries explicitly.

6.7 Lookup Semantics

For variable lookup of a symbol `s`:

1. search the active dynamic frames from innermost to outermost,
2. otherwise use the value cell of `s` in the current namespace,
3. otherwise, for explicit namespace functions only, use the designated target namespace,
4. otherwise treat the variable as unbound.

For function lookup of a symbol `s`:

1. use the function cell of `s` in the current namespace,
2. otherwise consult any explicit namespace import/export mechanism that the current namespace model provides,
3. otherwise treat the function as undefined.

This document does not yet define implicit fallback from one namespace class to another during ordinary function lookup.

6.8 Update Semantics

The following are the current design rules:

- `setq` updates a dynamic variable binding if one exists for the symbol in the active frame stack; otherwise it updates the value cell in the current namespace,
- `defun` updates the function cell of the symbol in the current namespace,

- `set-symbol-function` updates the function cell of the symbol in the current namespace,
- `vl-doc-set` updates a value cell in a document namespace designated by the namespace operation,
- `vl-bb-set` updates a value cell in the blackboard namespace,
- `vl-propagate` copies a variable value across document namespaces according to the documented session-wide propagation semantics.

Current conservative implementation:

- the runtime now provides explicit session/document/blackboard operations for document value lookup/update, blackboard lookup/update, and session-level propagation,
- propagation copies the current document value into all currently registered documents and also marks that symbol for copying into documents registered later in the same session,
- this is a runtime model for namespace mechanics, not yet a claim that the exact user-visible `vl-doc-*` / `vl-bb-*` product surface is fully matched.

Current conservative function-bridge rule:

- exporting a function from a separate VLX namespace copies its current function binding into the associated document namespace,
- importing a function copies a function binding from the associated document namespace into the current namespace,
- this models the documented document-associated bridge without yet claiming a complete product-accurate multi-application import model.

The interaction between `defun`, separate-namespace VLX code, and `vl-doc-export` / `vl-doc-import` remains a namespace-design task rather than a core symbol-design task.

7 Evaluator Model

The current evaluator is a direct tree-walking evaluator over runtime values.

7.1 Self-Evaluating Values

The following values currently evaluate to themselves:

- `nil`
- integers
- reals
- wrapped strings
- wrapped file and host-managed runtime objects

Symbols are not self-evaluating and are resolved through variable lookup.

7.2 Ordinary Calls vs Special Operators

Evaluation of a cons form currently proceeds as follows:

1. if the operator names a special operator, dispatch to the special-operator evaluator on the raw argument forms;
2. otherwise resolve the operator as a function binding in the current namespace, unless the operator is itself a `lambda` form;
3. evaluate ordinary call arguments from left to right;
4. apply the resulting callable object.

Current special operators implemented as evaluator primitives:

- `quote`
- `setq`
- `progn`
- `if`
- `cond`
- `and`
- `or`

- `while`
- `repeat`
- `foreach`
- `lambda`
- `function`
- `defun`

At this stage these are treated as fundamental evaluator cases, not as macro-like rewrites.

7.3 Function Application

For `SUBR` calls:

- the wrapped Common Lisp function is invoked on already evaluated AutoLISP runtime arguments,
- the active evaluation context is dynamically available while the `SUBR` runs, so namespace-sensitive builtins can resolve names against the current call context instead of against the process default context,
- any unexpected Common Lisp error is wrapped into an AutoLISP-visible runtime error rather than leaking directly.

For `USUBR` calls:

- the lambda list is split at `/` into required parameters and local variables,
- a new dynamic frame is pushed for the call,
- required parameters are bound to evaluated argument values,
- locals are initialized to `nil`,
- the body is evaluated with that frame active.

Function objects returned by `function` are treated as ordinary runtime values. At the current stage, `function` accepts:

- a symbol naming a function in the current namespace,
- a literal `lambda` form, which is converted to an anonymous `USUBR`.

7.4 BOUNDP

The runtime now models the documented `boundp` distinction as follows:

- an unbound symbol and a symbol explicitly bound to `nil` both yield `nil` as the visible `boundp` result,
- testing an unbound symbol with `boundp` materializes a namespace binding of that symbol to `nil`,
- therefore internal "has a value cell" state and visible `boundp` truth are intentionally distinct.

This preserves the Autodesk-documented behavior that an undefined symbol is created and assigned `nil` while `boundp` still returns `nil`.

7.5 DEFUN-Q Compatibility Definitions

The runtime now preserves a compatibility definition alongside ordinary function bindings for the `defun-q` family.

Current design rule:

- an ordinary `defun` stores only the callable `USUBR` in the function cell,
- `defun-q` stores both the callable `USUBR` and a list definition of the form `(lambda-list . body-forms)`,
- `defun-q-list-ref` returns that preserved list definition,
- `defun-q-list-set` replaces both the callable function binding and the preserved list definition.

This compatibility definition is attached to the namespace-local function cell, not to the symbol object itself.

7.6 Initial Error Hook and Catch-All Layer

The runtime now distinguishes two user-visible error bridges above raw `autolisp-runtime-error` conditions:

- a top-level `*error*` hook path,
- a Visual LISP catch-all object path.

Current rule:

- top-level runtime entry points such as file loading may trap an `autolisp-runtime-error` and, if a function named `*ERROR*` is defined in the current namespace, invoke it with one AutoLISP string argument carrying the message;
- if no `*ERROR*` handler is defined, the underlying runtime error is re-signaled;
- `vl-catch-all-apply` traps errors and returns a dedicated catch-all runtime object instead of unwinding;
- `vl-catch-all-error-p` and `vl-catch-all-error-message` operate on that dedicated object.

This is intentionally only the first layer, but the runtime now also exposes:

- a session-level `ERRNO` slot with conservative implementation-defined values,
- distinct termination conditions for `exit` and `quit`,
- a first namespace-exit bridge used by `vl-exit-with-error` and `vl-exit-with-value`.

Current additional rule:

- successful top-level execution resets `ERRNO` to `0`,
- handled runtime failures now use a conservative implementation-defined provisional classification:
  - `0` success,
  - `1` generic runtime failure,
  - `2` unbound variable,
  - `3` undefined function,
  - `4` invalid arguments, invalid syntax, or invalid call shape,
  - `5` file, stream, pathname, or load-surface failure,
  - `6` wrapped host/runtime bridge failure,
- `exit` and `quit` signal distinct runtime conditions so the eventual command loop can distinguish them,

- `vl-exit-with-error` and `vl-exit-with-value` signal dedicated namespace-exit conditions instead of leaking raw host conditions across namespace boundaries,
- control-transfer conditions such as `exit`, `quit`, and `vl-exit-*` are not ordinary trapped errors and must not be collapsed into Visual LISP catch-all error objects.

This table is deliberately provisional and implementation-defined. The exact Autodesk-compatible `ERRNO` code table, command-loop behavior, and full cross-namespace bridge semantics remain future work.

7.7 FOREACH

`foreach` is modeled as a special operator because it controls evaluation and introduces a temporary dynamic binding.

Current evaluation rule:

1. evaluate the list argument once,
2. push a dynamic frame,
3. iterate over the resulting proper list,
4. bind or update the iteration variable for each element in that frame,
5. evaluate the body forms for each element,
6. return the last body result, or `nil` if no body form is supplied.

8 Initial Runtime Type Set

This first runtime module should account for at least the spec-visible types:

- `nil`
- `INT`
- `REAL`
- `STR`
- `LIST`

- `SYM`
- `FILE`
- `ENAME`
- `PICKSET`
- `SUBR`
- `USUBR`
- `SAFEARRAY`
- `VARIANT`
- `VLA-object`

8.1 SUBR and USUBR

The runtime distinguishes builtin/native callable objects from AutoLISP-defined callable objects.

- `SUBR` builtin or primitive function implemented directly in Common Lisp
- `USUBR` AutoLISP-defined function evaluated by the `clautolisp` evaluator

Current design rule:

- a `SUBR` stores a name plus a Common Lisp function object,
- a `USUBR` stores a name, an AutoLISP lambda list, an AutoLISP body, and the runtime environment information needed by the evaluator,
- `defun` produces a `USUBR` bound in the current namespace,
- later anonymous `lambda` forms also produce `USUBR`-like function objects.

In practice this means:

- `SUBR` is used for builtin families such as arithmetic, list, string, file, and printer functions,

- `USUBR` is used for user-defined AutoLISP functions whose bodies must be evaluated form by form under AutoLISP dynamic binding rules.

Not all of these need complete behavior immediately, but their representation category should be known now.

9 Initial Error Model

The runtime now defines an initial AutoLISP-visible condition:

- `autolisp-runtime-error`

It currently carries:

- an error code,
- a human-readable message,
- optional structured details.

Current design rule:

- evaluator-detected errors should signal `autolisp-runtime-error`,
- invalid call shapes should use dedicated evaluator codes such as `:invalid-call-operator`, `:invalid-function-designator`, and `:invalid-function-object` instead of collapsing to a generic type error,
- unexpected Common Lisp errors caught while invoking a `SUBR` should be wrapped into `autolisp-runtime-error` with a dedicated `:subr-call-host-error` code,
- broader builtin-layer and host-layer error normalization still remains to be completed so the whole runtime presents a uniform AutoLISP-visible error surface.

10 Deferred Decisions

The following remain open:

- whether runtime string wrappers should retain source metadata from reading,

- whether all wrapped runtime objects share a common superclass or protocol,
- how `T` is bootstrapped and whether it is interned eagerly,
- how builtin functions and user functions differ structurally,
- how host adapters attach lifecycle and validity checks to wrapped objects.
- the exact operational rules by which host callbacks determine the current document and current namespace at entry,
- the exact semantics of separate-namespace VLX imports, exports, and calling contexts relative to document namespaces.
- the exact runtime object that holds current-directory and path-resolution state before the host API module is introduced.
- the exact pathname-resolution algorithm to implement once the specification corpus for file/path behavior is gathered and reconciled.